

# **SEEDCARE-RAG LoRA**

**Sistema multimodal para identificar posibles daños en semillas,  
recuperar métodos de prevención y manejo, y ampliar clases  
minoritarias mediante Stable Diffusion 1.5 ajustado con LoRA**

**Proyecto Integrador - ISIA-119 Modelos Generativos de IA**

Repositorio: rag-lora-stable-diffusion-seeds

Grupo 1: Sebastián Preciado Peralta, Carlos Mezones Burgos y Luis Reyes Guerrero

Julio de 2026

## 1. Resumen ejecutivo

SeedCare-RAG LoRA es un sistema de apoyo al control de calidad de semillas. Recibe una fotografía, estima si presenta un posible daño físico, biológico o morfológico mediante un modelo visual ajustado, recupera información técnica con una arquitectura RAG y genera un informe preliminar con fuentes, nivel de confianza, prevención, métodos de manejo y limitaciones. El proyecto incorpora entrenamiento generativo mediante un adaptador LoRA sobre Stable Diffusion 1.5, destinado a generar ejemplos sintéticos de clases minoritarias. Estas imágenes se revisan y se usan únicamente en el conjunto de entrenamiento.

## 2. Título final y descripción puntual

### Título final:

SeedCare-RAG LoRA: Sistema multimodal para identificar posibles daños físicos, biológicos y morfológicos en semillas, recuperar métodos de prevención y manejo, y ampliar clases minoritarias mediante Stable Diffusion 1.5 ajustado con LoRA.

### Descripción puntual:

La aplicación analiza una imagen de semilla, estima una categoría de daño visible y usa esa predicción para recuperar documentos especializados. Con el contexto recuperado, genera un informe técnico que diferencia observaciones, posibles causas y recomendaciones. En paralelo, Stable Diffusion 1.5 se ajusta con LoRA para crear datos sintéticos de categorías con pocas muestras, los cuales se incorporan solo después de revisión humana.

## 3. Problema y valor empresarial

- La revisión manual de semillas puede ser lenta y depender de la experiencia del evaluador.
- Las empresas pueden tener información dispersa en artículos, manuales y fichas técnicas.
- Las clases de daño poco frecuentes suelen tener pocos ejemplos para entrenar modelos.
- El sistema organiza el análisis preliminar, recupera evidencia y genera un informe trazable.

El sistema no debe presentarse como diagnóstico definitivo. Su valor es apoyar una inspección preliminar, priorizar muestras y facilitar el acceso a documentación relevante.

## 4. Arquitectura propuesta

| Módulo        | Función                                       | Salida                                |
|---------------|-----------------------------------------------|---------------------------------------|
| Dataset       | Auditar, etiquetar, separar y aumentar datos. | Metadatos y splits reproducibles.     |
| Modelo visual | Fine-tuning para clasificar daños visibles.   | Clase, probabilidades y checkpoint.   |
| SD 1.5 + LoRA | Generar ejemplos de clases minoritarias.      | Adaptador LoRA e imágenes revisables. |

|           |                                              |                                           |
|-----------|----------------------------------------------|-------------------------------------------|
| RAG       | Recuperar fragmentos de fuentes verificadas. | Top-k de documentos con scores.           |
| Informe   | Combinar predicción y evidencia documental.  | Informe preliminar con fuentes y límites. |
| Streamlit | Ofrecer la demostración en vivo.             | Interfaz de carga, análisis y reporte.    |

## 5. ¿Dónde se cumple el entrenamiento de modelos?

El requisito se cumple de forma explícita en dos partes. La primera corresponde al ajuste del modelo visual. La segunda es el entrenamiento generativo con LoRA sobre Stable Diffusion 1.5.

| Entrenamiento      | Qué se actualiza                                                                                  | Evidencia                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Fine-tuning visual | Capas del clasificador y, según el experimento, parte del backbone de ResNet18 o EfficientNet-B0. | Curvas de pérdida, checkpoint propio, macro-F1, métricas por clase y matriz de confusión. |
| LoRA de SD 1.5     | Matrices LoRA insertadas en el UNet; el modelo base permanece mayormente congelado.               | Archivo .safetensors, configuración, pérdidas, captions y muestras base frente a LoRA.    |

Experimento central: comparar el clasificador entrenado con datos reales contra el mismo clasificador entrenado con datos reales más imágenes sintéticas aceptadas. Esta comparación demuestra si el LoRA aporta una mejora medible o introduce ruido.

## 6. Metodología del dataset

1. Auditar archivos, resoluciones, duplicados, corrupción y distribución de clases.
2. Definir etiquetas principales: sana, daño físico, daño biológico y daño morfológico.
3. Marcar como no clasificada cualquier imagen dudosa hasta revisión.
4. Registrar especie, etiqueta, subetiqueta, procedencia, verificación y original\_id.
5. Separar 80% entrenamiento, 10% validación y 10% prueba antes de aumentar.
6. Aplicar aumento clásico únicamente al conjunto de entrenamiento.
7. Generar datos sintéticos solo para clases minoritarias y revisarlos manualmente.

## 7. Entrenamiento del modelo visual

- Línea base: ResNet18 preentrenado.
- Comparación opcional: EfficientNet-B0.
- Imagen de entrada: 224 x 224.
- Pérdida: cross-entropy; considerar pesos de clase si existe desbalance.

- Selección del mejor checkpoint por macro-F1 de validación.
- Test reservado hasta el final.

## 8. Entrenamiento de Stable Diffusion 1.5 con LoRA

8. Seleccionar imágenes reales verificadas de una o dos clases minoritarias.
9. Crear captions consistentes con el trigger word seedcare.
10. Construir una carpeta ImageFolder con metadata.jsonl.
11. Ejecutar una prueba técnica de 50 pasos.
12. Entrenar aproximadamente 800 pasos en Google Colab con GPU.
13. Guardar el adaptador LoRA, checkpoints y muestras de validación.
14. Generar varias imágenes por prompt y registrar seed y configuración.
15. Aplicar revisión humana de calidad y fidelidad de etiqueta.
16. Incorporar únicamente las imágenes aceptadas a train.

Configuración inicial: resolución 512, batch 1, gradient accumulation 4, learning rate 1e-4, rank 8 y precisión fp16. Estos valores son un punto de partida y deben ajustarse según la GPU y la estabilidad observada.

## 9. Construcción del RAG

17. Reunir artículos, manuales y fichas con procedencia verificable.
18. Extraer texto y registrar título, institución, año y temas.
19. Dividir el texto en fragmentos con solapamiento.
20. Crear embeddings multilingües y un índice FAISS.
21. Construir la consulta con la clase estimada y observaciones visuales.
22. Recuperar top-k fragmentos y mostrar fuente, score y texto.
23. Generar el informe usando solo el contexto recuperado y una advertencia explícita.

## 10. Métricas y experimentos

| Componente       | Métricas principales                                                        |
|------------------|-----------------------------------------------------------------------------|
| Modelo visual    | Accuracy, macro-F1, precision/recall por clase, matriz de confusión.        |
| LoRA             | Pérdida, calidad visual 1-5, fidelidad de etiqueta 1-5, tasa de aceptación. |
| Efecto sintético | Macro-F1 y F1 minoritaria antes/después de incorporar sintéticos.           |
| RAG              | Precision@k, Recall@k, MRR, respaldo de afirmaciones y latencia.            |
| Sistema          | Tiempo total, claridad del informe y presencia de fuentes/limitaciones.     |

## 11. Uso de Codex local

Codex se utilizará para implementar tareas delimitadas dentro del repositorio, ejecutar pruebas y revisar cambios. Los prompts se encuentran en la carpeta codex/. Cada tarea debe trabajarse en una rama separada, con revisión del diff antes del commit. Codex no debe inventar métricas, eliminar datos originales ni incluir credenciales.

- Primero inspeccionar y planificar.
- Una tarea por prompt y rama.
- Ejecutar pruebas antes de entregar.
- Revisar cambios humanos antes del merge.
- No delegar decisiones de etiquetado dudoso sin supervisión.

## 12. Flujo de GitHub

24. Crear el repositorio privado rag-lora-stable-diffusion-seeds.
25. Subir esta estructura a main como versión inicial.
26. Crear ramas feature/dataset-audit, feature/vision-training, feature/lora-training, feature/rag y feature/app.
27. Realizar commits pequeños y abrir Pull Requests.
28. Requerir revisión de otro integrante antes de fusionar.
29. Mantener datasets, pesos y credenciales fuera de Git.

## 13. Cronograma sugerido

| Día | Trabajo                                | Evidencia                                    |
|-----|----------------------------------------|----------------------------------------------|
| 1   | Repositorio, auditoría y etiquetas.    | summary.json, images.csv y distribución.     |
| 2   | Split, aumento clásico y línea base.   | Metadatos con split y primer entrenamiento.  |
| 3   | Fine-tuning visual.                    | Checkpoint y métricas.                       |
| 4   | Preparación y entrenamiento LoRA.      | Adaptador y muestras.                        |
| 5   | Filtrado sintético, comparación y RAG. | Tabla antes/después e índice.                |
| 6   | Integración, ética y pruebas.          | Demo funcional y resultados.                 |
| 7   | Informe IEEE y sustentación.           | Documento, diapositivas y video de respaldo. |

## 14. Riesgos y contingencias

- Etiquetas insuficientes o ambiguas: reducir subclases y mantener cuatro categorías principales.

- LoRA de baja calidad: conservar comparación honesta y usar solo imágenes aceptadas.
- GPU de Colab no disponible: guardar checkpoints, muestras y video de respaldo.
- RAG con fuentes débiles: priorizar documentos oficiales, artículos y manuales con procedencia.
- Clasificación errónea: mostrar probabilidades y permitir resultado no concluyente.

## 15. Reflexión ética

El proyecto debe comunicar que una imagen no confirma una enfermedad. Debe evitar recomendaciones categóricas, informar las limitaciones del dataset y registrar la procedencia de los documentos. Las imágenes sintéticas deben identificarse como tales y no usarse como evidencia real. La supervisión humana es obligatoria para revisar etiquetas, resultados y recomendaciones.

## 16. Entregables finales

- Repositorio GitHub con README e instrucciones de ejecución.
- Dataset documentado y metadatos reproducibles.
- Checkpoint del clasificador visual.
- Adaptador LoRA de Stable Diffusion 1.5.
- Índice RAG y conjunto de fuentes documentadas.
- Aplicación Streamlit demostrable.
- Métricas, tablas y figuras.
- Informe IEEE, presentación y plan de contingencia.

## 17. Referencias técnicas sugeridas

- Documento de alcance del Proyecto Integrador ISIA-119.
- Documentación oficial de PyTorch sobre transfer learning para visión.
- Documentación oficial de Hugging Face Diffusers y script `train_text_to_image_lora.py`.
- Documentación de Transformers para modelos de embeddings y CLIP.
- Documentación oficial de FAISS para búsqueda de similitud.
- Documentación oficial de OpenAI Codex para flujos locales y revisión de repositorios.